Trier University
Department IV – Computer Sciences
Prof. Dr.-Ing. Ralf Schenkel, Martin Blum (blumma@uni-trier.de)

Trier, May 6, 2026

Assignment 3 of the course Big Data Analytics

Summer semester 2026

Deadline 10:15h on Wednesday, May 20, 2026

General remarks on programming for Hadoop

In StudIP you can find information on how to setup Hadoop in Eclipse and IntelliJ. In the same directory, you can find the WordCount example from the lecture.

You do not have to run your solutions on a real Hadoop cluster. It is sufficient if you run your programs in the stand-alone mode.

Should you encounter problems with setting up the framework, please send a mail to Martin Blum.

Task 1: WordCount++ 15 points

Modify the `WordCount` example such that not the number of overall occurrences is counted, but the number of lines in which a term occurs. Additionally, only terms that occur in **at least 5 lines** should be returned.

- Compared to the original `WordCount`, modify the Mapper implementation as follows. Instead of directly writing the tokens retrieved from the `StringTokenizer`, store them in an appropriate data structure like a `Set`. After all tokens were consumed, output the tokens included in the set. You can use a for-each loop for this, for example.
- In the Reducer implementation, you need to check if the number of lines exceeds the given threshold before you output the result for a term.

Hand in your code and the output of the reducer for the example input file `small_corpus.txt` that can be found in Moodle.

Task 2: Grep 15 points

Implement the grep example from slide 31 of Chapter 3 as a Hadoop MapReduce application.

- You can use the `WordCount` example as a template.
- The input to this task should be the file `corpus_with_line_numbers.txt` provided in Moodle (you need to decompress it first). In this file, every line corresponds to one document. A line has the form

  ```
  docid: text...
  ```

  where `docid` is a unique integer greater than 0, and `text` is the text of the document.
- You can either read the expression to search for from the command line or define it in your source code.

- Write a corresponding implementation of the `Mapper` interface (like in the `WordCount` example). Extract the docid and the text from the line read. If the text includes the searched expression, emit the docid as a result of the mapper; otherwise, emit nothing. Use an appropriate Java function for identifying matching text.
- Since this is a Map-only job, there is no need for a reducer. You need to set this in the Job configuration in the `run()` method, using `job.setNumReduceTasks(0);`

Hand in your code and the result of grepping for the term `"malware"` (without the quotes) in the provided file.

Task 3: Inverted Index 15 points

Write an Hadoop MapReduce application that generates an inverted index for a document collection, following slides 37 and 38 from Chapter 3 of the lecture.

- You can use the `WordCount` example as a template.
- The input to this task should be the file `small_corpus_with_line_numbers.txt` provided in Moodle. In this file, every line corresponds to one document. A line has the form

  ```
  docid: text...
  ```

  where `docid` is a unique integer greater than 0, and `text` is the text of the document.
- Write a corresponding implementation of the `Mapper` interface (like in the `WordCount` example). Extract the docid and the text from the line read. Split the text into its tokens using a `StringTokenizer`. For each word, emit the word as a key and the docid as a value (converted to an `int`). Write a corresponding implementation of the `Reducer` interface (like in the `WordCount` example). Your reducer will emit `Text` as key (for the word) and `Text` as value (for the list), so you need to adapt the generics:

  ```
  public static class Reduce
                      extends Reducer<Text, IntWritable, Text, Text>
  ```

  In the reducer, build a String representation of the corresponding list. For example, if you read the numbers 1,2, and 5 from the `Iterable`, the String should look as follows:

  ```
  1, 2, 5
  ```

  Write that String as a result, using the `write` method of the `context` object. Instead of an `IntWritable`, you need a `Text` object since we are writing a String, not a number.
- In order to reduce the number of written lists, **emit the list only if it contains at least five entries**. For shorter lists, emit nothing.
- Adapt the job configuration such that the result value of the reducer is of type `Text`, not `IntWritable`:

  ```
  job.setOutputValueClass(Text.class);
  ```

  in the `run()` method. Since our job now has different keys and values for Map and Reduce, we also need to specify the corresponding classes for the Map task:

  ```
  job.setMapOutputKeyClass(Text.class);
  job.setMapOutputValueClass(IntWritable.class);
  ```

Hand in your code and the output of the reducer for the example input file.

These tasks will be discussed on June 1, 2026 (since May 25 is a regional holiday).

General remarks:

- In general, the tutorial group takes place on Mondays at 14:15 and 16:15 in HS 11 on a (roughly) bi-weekly basis.
- On May 4 and May 11, there will no meeting at 16:15.
- To be admitted to the final exam, you need to acquire at least 50% of the points in the assignments.
- It is required to submit in groups of size 3; only one submission is sufficient for the whole group. Groups must be chosen in Moodle (see link on the course page in Moodle). Write the names of all group members on your solutions. Students without a group cannot submit.
- Solutions must be handed in before the deadline in Moodle (`https://moodle.uni-trier.de/`, course `BDA-26`). Submissions that arrive after the deadline will not be considered.
- Source code has to be submitted in a plain text format which allows us to execute and test your code. For other files you may use the PDF format. Combine multiple files in a ZIP file or comparable.
- Graded versions of your submissions will be returned in Moodle until the following tutorial.
- Announcements regarding the lecture and the tutorial group will be done in the area of the lecture in StudIP.